

AI Agents

生成式人工智能工具整合與工作流程應用

Prompting Guide: Agents

Workflows vs Agents

Function Calling

Context Engineering

本課重點：把 LLM 從「回答問題」設計成「可控、可追蹤、可覆核的任務系統」。

本節學習目標

完成本節後，學員應能：

- 說明 AI agent 與一般 LLM prompt 的差別
- 判斷何時應使用固定 workflow，何時才需要 agent
- 解釋 agent 的核心組件：planning、tools、memory、context、verification
- 看懂 function calling 如何讓 LLM 呼叫外部工具
- 為警務文件處理場景設計一個可審計的 agent 流程

警務應用中，agent 是「決策支援與流程輔助」，不是法律結論、紀律處分或證據判斷的自動替代。

建議學習路線

1. Agent

先定義它不是 chatroom，而是任務系統

2. Workflow

先學可控流程，再判斷是否需要自主 agent

3. Tools

用 function calling 連接外部能力

4. Context

把規則、資料、狀態和工具說明放對位置

5. Deep Agent

長任務需要 planning、memory、delegation、verification

設計順序：不要一開始就追求「全自主」。先把任務拆成可觀察 workflow，再逐步加入 agentic decision。

Topic 1 | What Is an AI Agent?

從「生成答案」到「完成任務」

核心問題：如果 LLM 需要自己決定下一步、呼叫工具、記住狀態，它就不再只是 prompt，而是 agentic system。

什麼是 AI Agent ?

Prompting Guide 將 agent 描述為由 LLM 驅動、能採取行動並自主處理複雜任務的系統。

它通常具備：

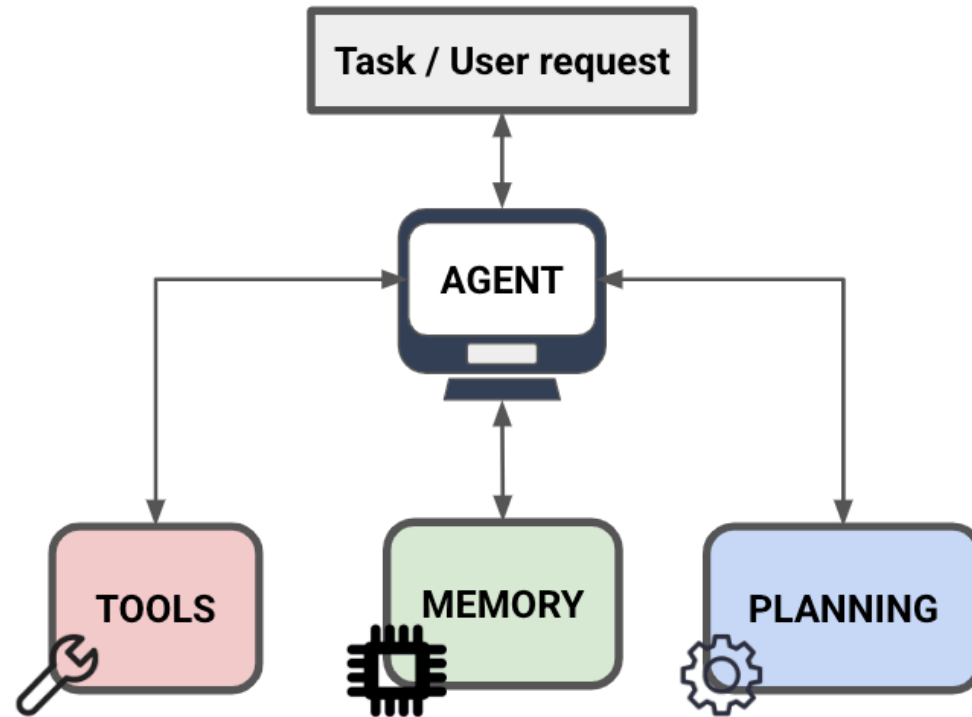
- **Planning**：拆解任務、決定下一步
- **Tool Use**：呼叫搜尋、資料庫、API、文件系統
- **Memory**：保存任務狀態與過往結果
- **Reflection / Verification**：檢查輸出、重試或升級人工覆核

警務例子

處理案件摘要時，agent 可以：

- 讀取案件摘要
- 判斷需要查哪些內部規程或法例
- 產生查詢詞並等待搜尋結果
- 整理已核對與未核對事項
- 產生報告草稿與風險提示

Agent 的核心組件



Prompting Guide: Agent Components

一般 Prompt vs Agent

比較面向	一般 Prompt	AI Agent
任務形態	一次輸入，一次輸出	多步驟、可迭代
流程控制	使用者手動控制	系統可規劃與選下一步
資訊來源	prompt 內文字與模型知識	工具、資料庫、搜尋、文件
狀態管理	通常沒有	task state、memory、tool log
錯誤處理	使用者重新問	可重試、記錄、請求人員確認
風險	幻覺較難追蹤	工具與日誌可提升可審計性，但設計更複雜

不要把所有 LLM 應用都叫 Agent

不是 agent 的例子

- 摘要一份報案筆錄
- 將公文改寫成較清晰語氣
- 根據固定 rubric 做一次分類
- 根據提供資料生成表格

這些通常是 **single call** 或 **prompt chain**。

接近 agent 的例子

- 先判斷資料不足，再要求補件
- 選擇使用法例搜尋或內部 SOP 查詢
- 根據搜尋結果決定是否再查另一組關鍵詞
- 在長任務中維持進度與待辦清單

這些有 **自主決策與工具選擇**。

Topic 2 | Workflows vs Agents

先講可控 workflow，再談 autonomous agent

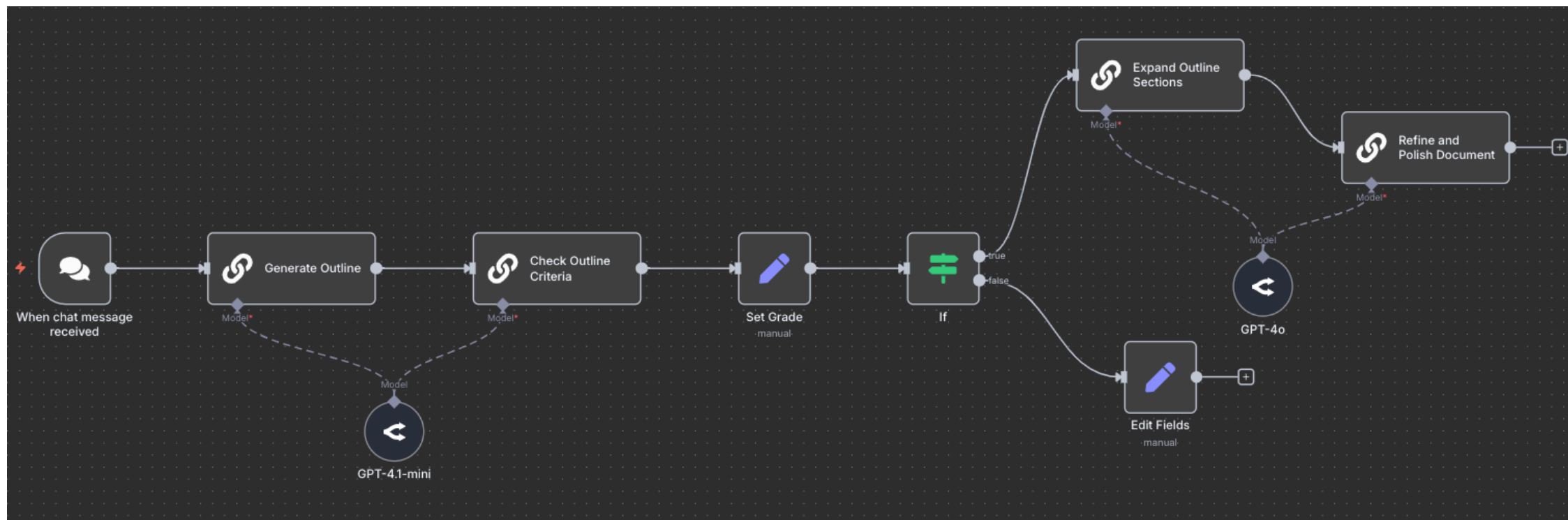
Prompting Guide 的重點：workflow 和 agent 都可使用 LLM，但 autonomy 的程度不同；工程設計要根據風險和任務穩定性選擇。

Workflow 與 Agent 的核心差別

面向	AI Workflow	AI Agent
控制方式	開發者預先定義步驟	LLM 動態決定下一步
適合任務	規則清楚、流程穩定	開放式、多變、需探索
可預測性	高	較低
審計難度	較低	較高，需要日誌與觀察
警務適用	表格抽取、固定報告、標準檢核	多來源資料整理、策略性搜尋、長任務支援

高風險、法定程序、個資處理，應優先使用 workflow 或 human-in-the-loop agent。

Pattern 1 : Prompt Chaining



把大任務拆成多個可檢查的小步驟。

Prompt Chaining：警務文件例子

抽取

人物、時間、地點、事件、
證物

分類

案件類型、風險等級、所需
表格

檢核

資料缺口、矛盾、待查事項

查詢

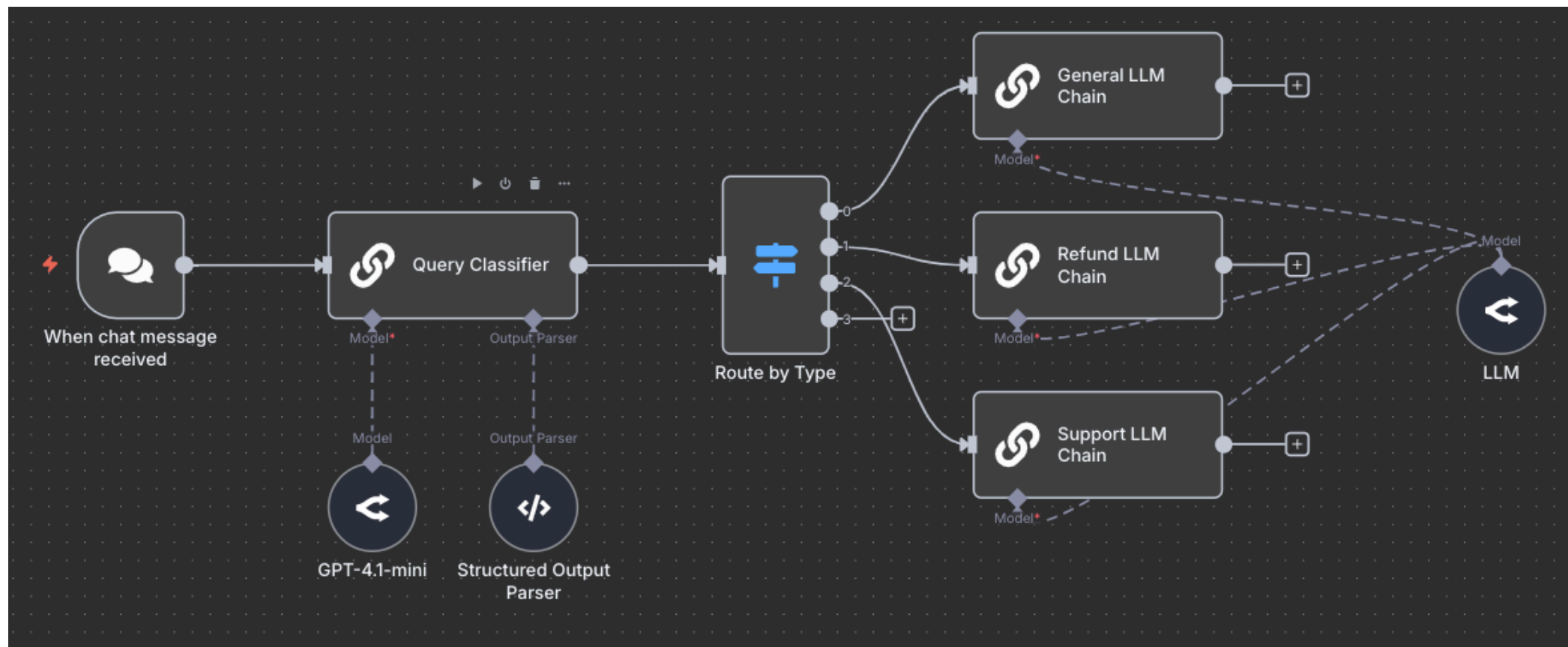
生成法例或 SOP 搜尋詞

草擬

只根據已核對資料生成報告
草稿

優點：每一步都有輸入、輸出與檢查點，適合教學、審計和逐步改良。

Pattern 2 : Routing

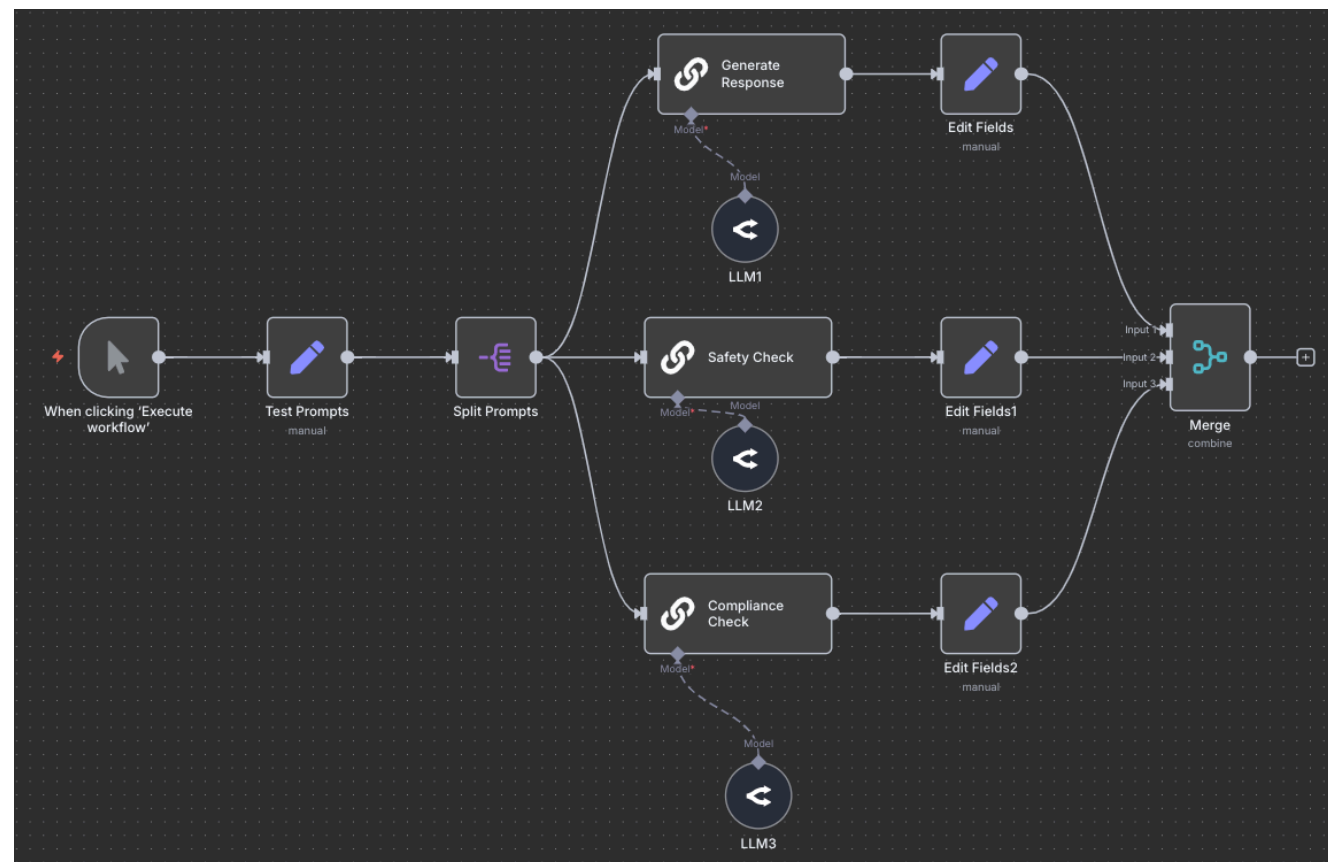


先判斷任務類型，再把資料送到合適處理路徑。

Routing：警務處理場景

輸入類型	路由到	產出
投訴信	投訴分類 workflow	涉及單位、重點事實、待核對事項
報案摘要	案情結構化 workflow	人時地事物、缺漏、初步分類
法例查詢需求	法例搜尋助理	搜尋詞、搜尋目的、預期核對內容
內部 SOP 問題	RAG / knowledge base	引用來源的程序提示
多文件材料	Agentic review	先建立任務清單，再逐項核對

Pattern 3 : Parallelization



同一任務可分派多個平行檢查，最後再整合。

Parallelization：何時有用？

多角度檢核

- 事實一致性
- 時序完整性
- 資料缺口
- 語氣與格式

多來源整理

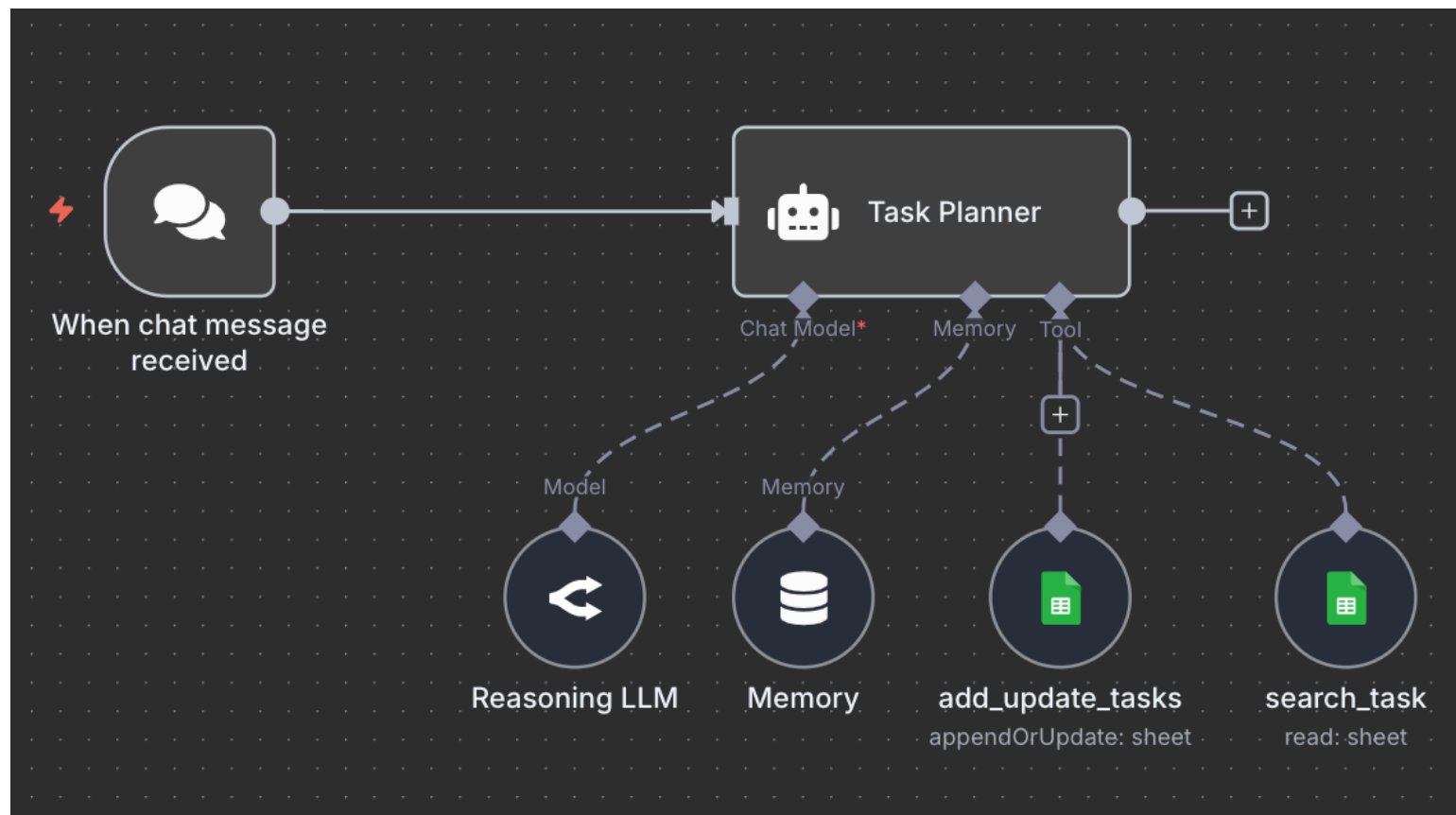
- 案件摘要
- 內部指引
- 公開法例
- 既有模板

多角色覆核

- 報告撰寫
- 風險提示
- 法例搜尋
- 人員審批

平行不是為了更炫，而是為了將不同檢查標準分開，減少單一 prompt 混雜。

Agentic Workflow : Task Planner



當任務需要動態拆解與追蹤時，workflow 開始接近 agent。

何時才需要 Agent ?

使用 Workflow

- 步驟穩定、可預先定義
- 輸出格式固定
- 風險高，需要容易審計
- 不需要 LLM 自己選工具

例：案件摘要轉結構化表格

使用 Agent

- 任務需要探索
- 資料來源多且不確定
- 需要根據工具結果調整下一步
- 任務時間長，需要保存狀態

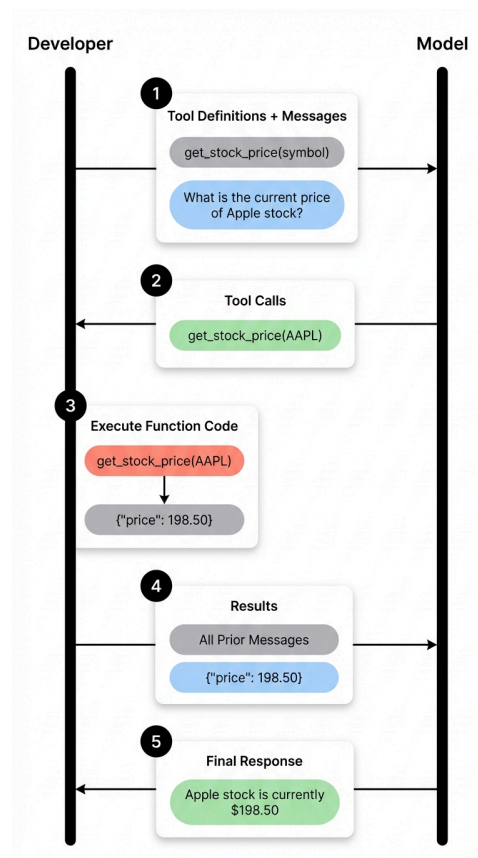
例：多文件案件初步整理與查漏

Topic 3 | Function Calling and Tools

Agent 為何能「做事」？

Function calling 讓 LLM 不只是輸出文字，而是根據工具 schema 產生可執行的工具呼叫參數。

Function Calling Flow



模型選擇工具與參數，系統執行工具，再把 observation 放回脈絡。

Tool Definition 要說清楚什麼？

工具定義通常包含：

- **name**：工具名稱
- **description**：何時應使用
- **parameters**：需要哪些欄位、型別、限制
- **return shape**：回傳內容如何解讀
- **safety rules**：何時不能自動執行

重點

工具描述會直接影響模型是否選對工具。

如果 description 太含糊，agent 可能：

- 用錯工具
- 漏用工具
- 用錯參數
- 把工具結果當成已證實結論

警務例子：法例搜尋工具

```
{
  "name": "macau_law_search",
  "description": "Generate or submit search queries to the Macau law search system. Use when a case summary needs legal text lookup. Do not provide legal conclusions.",
  "url": "https://search.bo.dsaj.gov.mo/?lang=zh-mo&scope=full_text&type=all",
  "parameters": {
    "query": "string",
    "purpose": "string",
    "expected_checks": "string"
  }
}
```

工具可以有 URL，但 agent 的權限要分清楚：是「只產生搜尋詞」、還是「真的送出查詢並讀取結果」。

工具呼叫不等於法律結論

階段	Agent 可做	Agent 不應做
搜尋前	生成查詢詞、列搜尋目的	假設某條法例一定適用
搜尋中	呼叫搜尋工具、記錄 query	改寫或編造搜尋結果
搜尋後	整理貼回來的結果、列待核對點	直接下法律判斷
報告草稿	標示「依據來源」和「未核對」	把未核對資料寫成確定事實

Tool Log：可審計性的最低要求

每次 agent 使用工具，至少記錄：

- 使用者輸入與任務 ID
- 被選中的工具名稱
- 工具參數
- 工具回傳 observation
- agent 如何根據 observation 決定下一步
- 哪些內容需要人員覆核

對教學而言，tool log 是最重要的 debug 材料；對警務應用而言，它是問責與審計材料。

Topic 4 | Context Engineering

Agent 好不好，不只看 prompt，而看整個 context

Prompting Guide 強調：context engineering 是設計、測試、迭代 agent 所接收的脈絡資訊，包括指令、限制、工具描述、記憶和錯誤處理。

Prompt Engineering vs Context Engineering

面向	Prompt Engineering	Context Engineering
主要對象	單次 LLM 呼叫	多步驟 agent system
關注點	問法、格式、例子	指令、資料、工具、記憶、狀態、錯誤
成功標準	回答更好	行為更穩定、可追蹤、可重現
常見問題	回答不完整	漏步驟、錯用工具、狀態丟失
改良方法	改 prompt	改 context layer、log、tool schema、validation

Context 包含哪些東西？

Agent 的 context 可以分成四層：

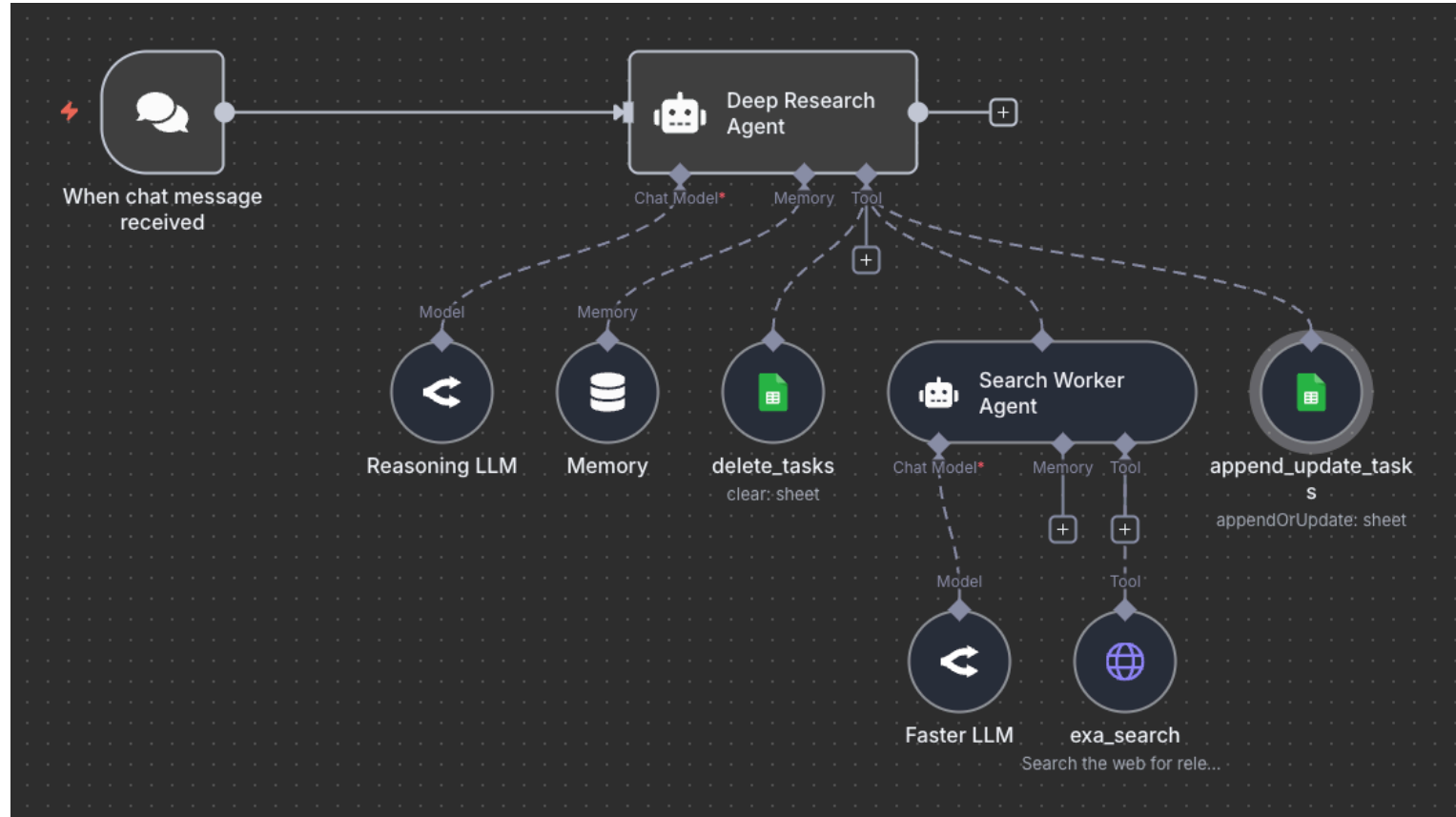
1. **System Layer**：角色、權限、禁止事項
2. **Task Layer**：當前任務、輸入、輸出格式
3. **Tool Layer**：工具說明、參數、使用限制
4. **Memory Layer**：已完成步驟、歷史結果、待辦事項

警務文件處理

Context 應包括：

- 案件摘要與來源標籤
- 可用工具清單
- 查詢結果與引用片段
- 內部 SOP 摘要
- 未核對事項
- 人工覆核要求

Context Engineering Case : Deep Research Agent



Prompting Guide 用 deep research agent 說明：如果 context 沒說清楚，agent 可能漏做任務。

常見失敗：Agent 靜悄悄跳過步驟

問題

Agent 建立了三個搜尋任務，卻只執行兩個。

沒有說明為何跳過第三個，也沒有把任務標示為取消或完成。

改法

明確規定：

- 每個 task 必須執行或說明為何不執行
- 合併重複任務前要先記錄理由
- 每次工具呼叫後更新 task tracker
- 無法處理時要請求人員確認

Context Template：警務 Agent 可用格式

SYSTEM:

你是警務文件處理助理。你只做整理、檢索、草擬和風險提示。
不得作法律結論、不得編造來源、不得隱瞞未核對事項。

TASK:

根據案件摘要，建立「待查事項清單」和「工具使用計劃」。

TOOLS:

- macau_law_search: 只產生或送出法例搜尋查詢
- internal_sop_search: 查詢內部程序摘要
- report_template: 生成指定格式草稿

STATE:

- 已完成:
- 待處理:
- 需要人工確認:

Context Engineering 的檢查清單

檢查項	問題
Completeness	是否覆蓋主要任務、例外和錯誤情況？
Clarity	工具何時用、何時不用是否清楚？
Consistency	system、tool、task 的規則有沒有互相矛盾？
Observability	是否記錄中間決策、工具參數與結果？
Testability	是否可以用測試案例驗證 agent 有沒有漏步？

Topic 5 | Deep Agents

長任務需要 **planning、memory、delegation、verification**

Deep agents 不是「更長的 prompt」，而是為長任務設計的 agent architecture。

Deep Agent 的核心概念

淺層 agent 容易在長、多步驟任務中失敗；deep agent 透過規劃、記憶、分工和驗證來處理更複雜的任務。

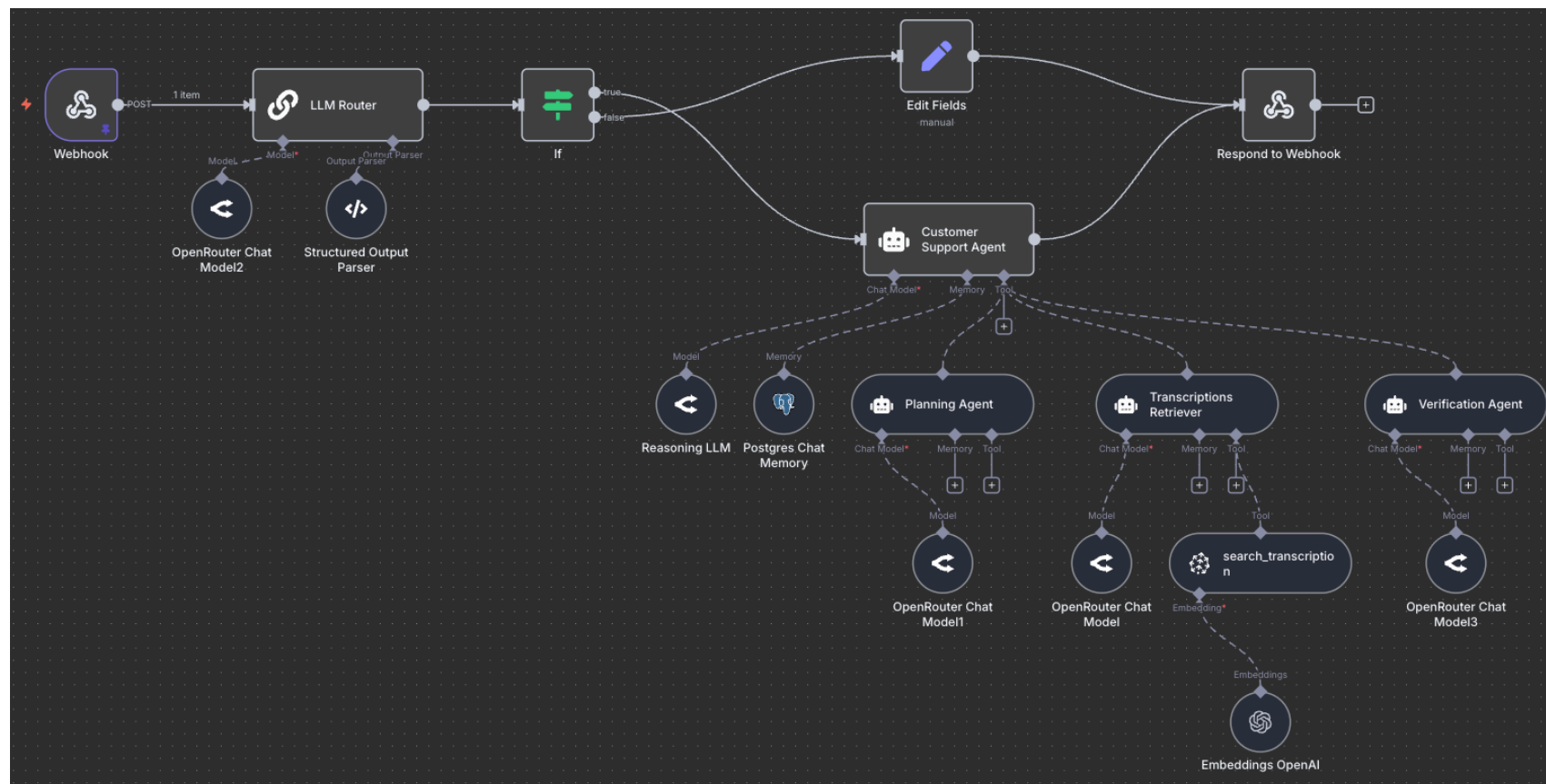
需要 deep agent 的任務

- 深度研究
- 多文件審閱
- agentic coding
- 長時間客服或知識庫支援

警務可類比場景

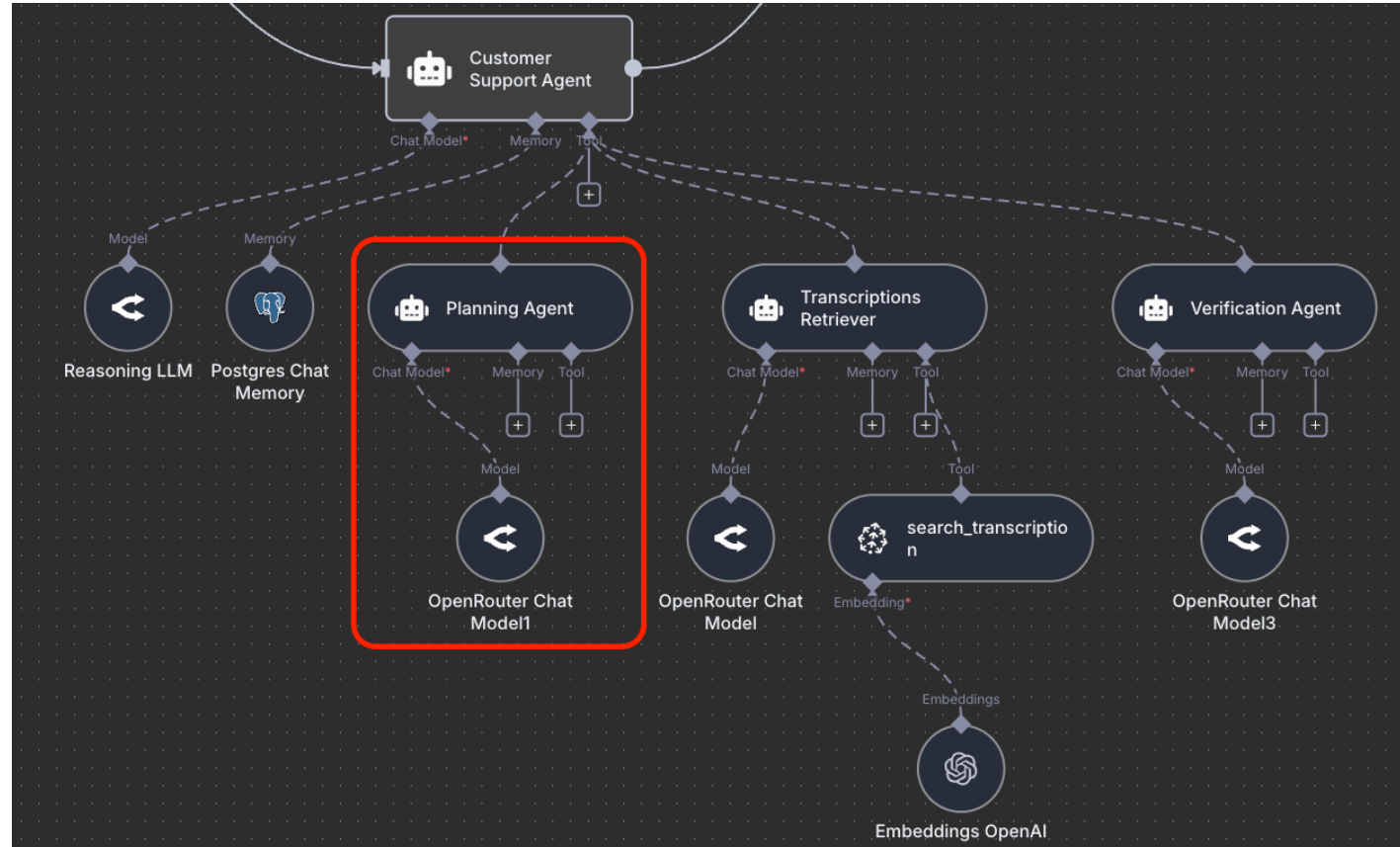
- 多文件案件整理
- 長篇口供與證物對照
- 投訴材料初步分類與查漏
- 內部規程 + 公開法例交叉檢索

Deep Agent Example



Prompting Guide 的 deep agent 例子：以 customer support agentic RAG system 展示長任務架構。

Component 1 : Planning



Deep agent 維持可更新、可重試、可恢復的 task plan。

Planning：警務場景設計

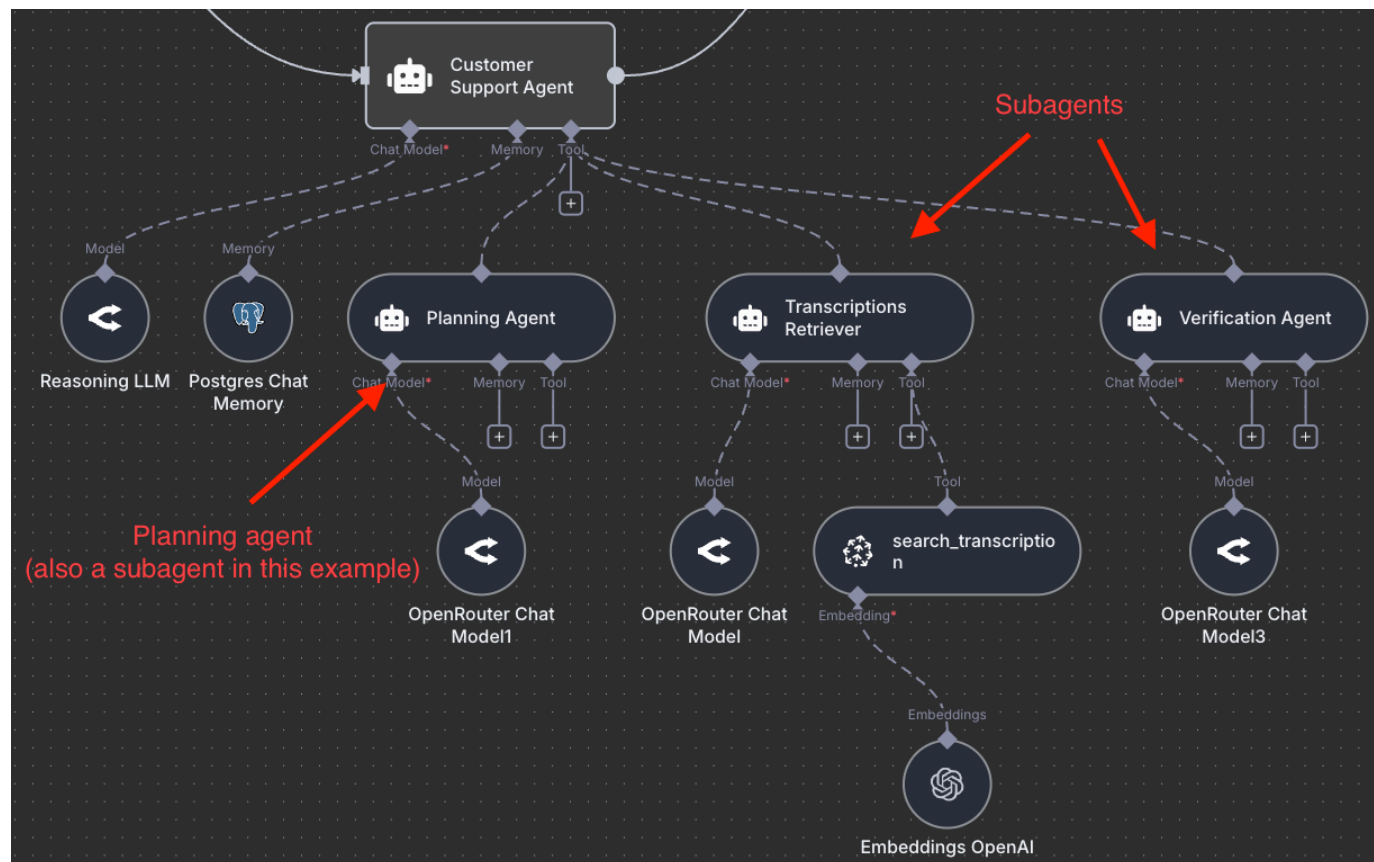
任務：整理一組案件材料並產生主管覆核摘要

Plan:

1. 建立材料清單與來源標籤
2. 抽取人、時、地、事、物
3. 找出資料缺口與矛盾
4. 生成法例搜尋與 SOP 查詢計劃
5. 等待搜尋結果或人工補充
6. 只根據已核對資料生成摘要草稿
7. 列出不得自動判斷的事項

Plan 必須可見、可修改、可審計。

Component 2 : Orchestrator + Sub-agents

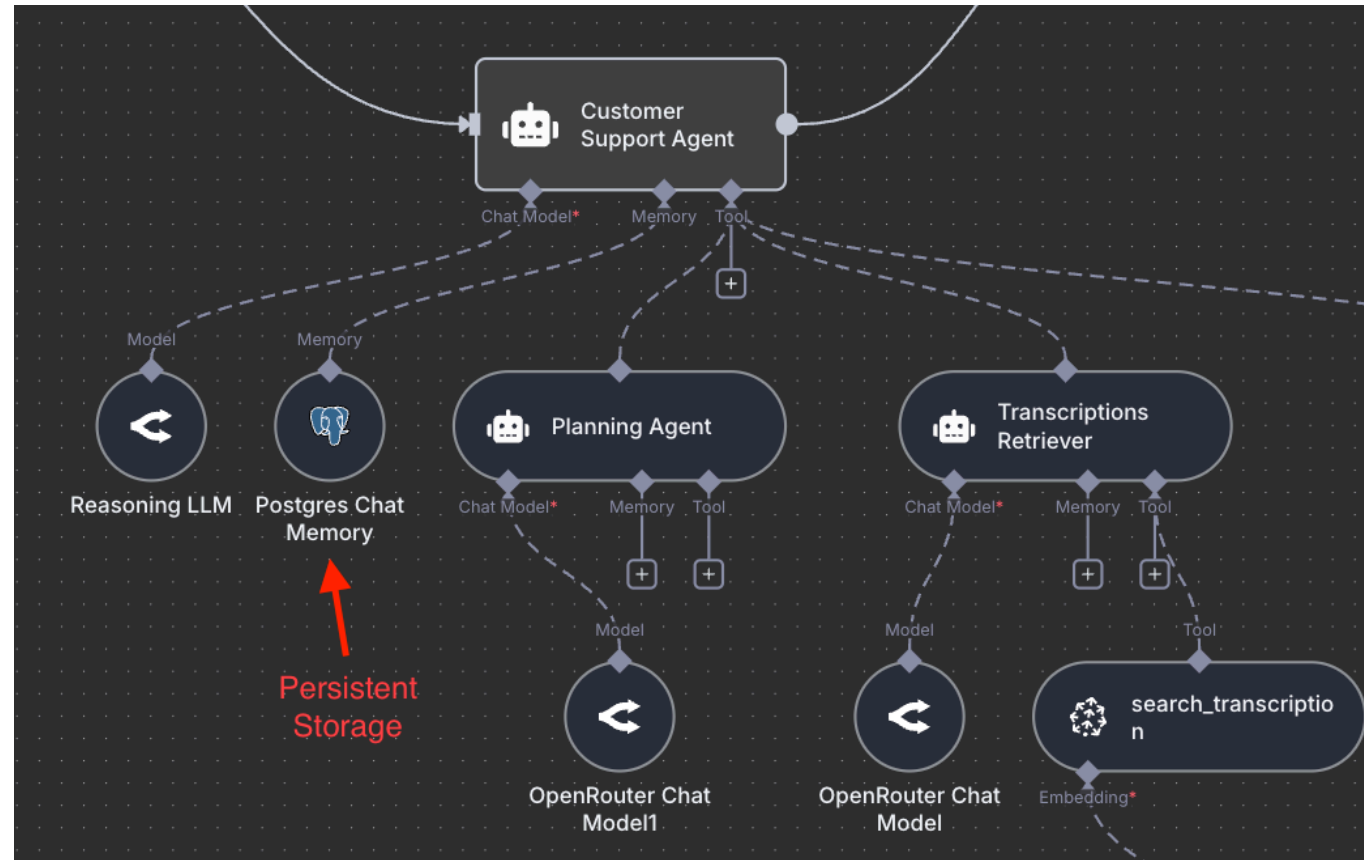


Orchestrator 負責分派任務；sub-agent 保持較乾淨、較專門的 context。

Sub-agent 可以怎樣分工？

Sub-agent	職責	不應做
Extractor	抽取結構化事實	推測未提供事實
Search Planner	產生法例 / SOP 查詢	直接下法律結論
Consistency Checker	找時序與陳述矛盾	自行修正原文
Draft Writer	生成報告草稿	隱藏未核對事項
Verifier	檢查來源與格式	代替主管批准

Component 3 : Context Retrieval and Memory



Deep agent 不只依賴聊天紀錄，而是把中間成果放入外部記憶或任務文件。

Memory：不要把所有東西塞進對話

短期狀態

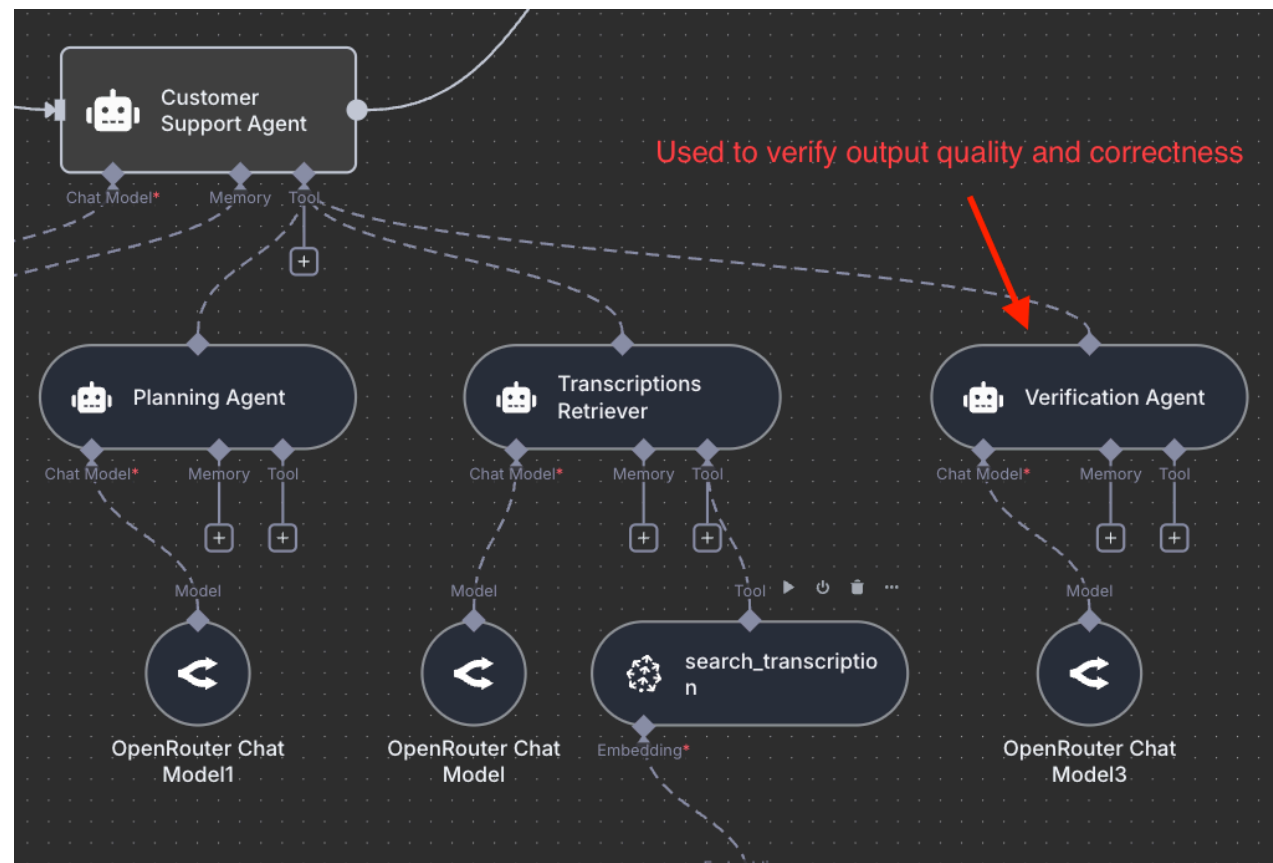
- 當前 task
- 最近 tool observation
- 待回答問題
- 下一步行動

長期記憶

- 已核對資料清單
- 常用報告格式
- 內部知識庫引用
- 過往任務的錯誤模式

長期記憶涉及個資與敏感資料時，必須有保存期限、權限控制、刪除機制與審計紀錄。

Component 4 : Verification



Verification 可以由模型輔助，但高風險結論仍要由人員覆核。

Verification：警務文件最低標準

檢查面向	檢查問題
Source grounding	每個關鍵事實是否有來源？
Missing information	是否列出未提供或未核對資料？
Tool trace	是否記錄搜尋詞、工具結果與使用時間？
Legal risk	是否避免把搜尋結果寫成法律結論？
Human review	哪些段落需要主管或法律專業人員確認？

Topic 6 | Police Processing Design

把 agent 概念落到警務文件與資料處理

本節不追求「全自動」，而是設計一個可控、可審計、可逐步升級的警務文件處理 agent。

一個合理的警務 Agent 架構

Input

案件摘要、附件、資料來源
標籤

Plan

建立待辦、資料缺口、工具
計劃

Tools

法例搜尋、SOP 檢索、模板
生成

Draft

只根據已核對資料生成草稿

Review

列出風險、未核對點、人工
覆核

關鍵不是讓 agent 自己「判案」，而是讓它把資料整理、查詢與覆核流程做得更透明。

課堂練習：Workflow 還是 Agent？

請判斷下列任務應用 workflow 還是 agent，並說明理由：

任務	建議
將報案摘要轉成固定欄位表格	Workflow
根據案情自動產生多組法例搜尋詞，再等待結果整理	Agent / human-in-loop
按固定 rubric 檢查報告格式	Workflow
多文件材料中找出矛盾、缺漏，再決定下一步查詢	Agent
根據內部 SOP 回答「下一步程序」	RAG workflow，必要時加 agent

課堂練習：設計一個工具

為以下工具補齊 description、parameters 和 safety rules：

```
{
  "name": "internal_sop_search",
  "description": "...",
  "parameters": {
    "query": "...",
    "department": "...",
    "purpose": "..."
  },
  "safety_rules": ["..."]
}
```

要求：

- 明確說明何時應使用
- 明確說明不得輸出未核對程序
- 回傳內容必須保留來源標籤

課堂練習：改寫成 Context

原始 prompt：

幫我看這個案件摘要，告訴我下一步要做什麼。

請改寫成 agent context，至少包含：

- agent 角色與禁止事項
- 任務目標
- 可用工具
- 輸出格式
- 不確定時的處理方式
- 哪些內容需要人工覆核

Design Principle：警務 Agent 的紅線

可以做

- 摘要、分類、抽取
- 產生搜尋詞
- 建立待辦清單
- 查漏與格式檢查
- 草擬可覆核文件

不應自動做

- 法律結論
- 證據真偽判斷
- 強制措施建議
- 個案最終處理決定
- 無日誌的外部查詢

本節總結

- Agent 是可規劃、可使用工具、可保存狀態的任務系統
- Workflow 較可控，agent 較靈活；警務場景應先選可審計方案
- Function calling 是 agent 連接外部工具的常見方式
- Context engineering 決定 agent 是否穩定、可追蹤、可測試
- Deep agents 適合長任務，但必須加入 planning、memory、delegation、verification
- 警務應用的底線是 human-in-the-loop、來源透明與工具日誌

References

- Prompt Engineering Guide: Agents
<https://www.promptingguide.ai/agents>
- Introduction to AI Agents
<https://www.promptingguide.ai/agents/introduction>
- Agent Components
<https://www.promptingguide.ai/agents/components>
- AI Workflows vs. AI Agents
<https://www.promptingguide.ai/agents/ai-workflows-vs-ai-agents>
- Function Calling in AI Agents
<https://www.promptingguide.ai/agents/function-calling>
- Why Context Engineering?
<https://www.promptingguide.ai/agents/context-engineering>
- Deep Agents
<https://www.promptingguide.ai/agents/deep-agents>